

1. **Scenario 1: Pointers to Class Members** You're developing a vehicle management system with a class `Vehicle` that has several data members (for example, an integer `speed` and a float `fuelLevel`) and a member function `displayStatus()`. Parts of the system need to dynamically modify and display these attributes using pointers to class members. (a) How would you declare a pointer to the speed data member and a pointer to the member function `displayStatus()`? (b) Provide a code snippet that uses these pointers to set the speed of a `Vehicle` object and then calls `displayStatus()`. (c) What are some potential pitfalls when using pointers to class members, and how can they be mitigated?

Ans. a. `int Vehicle: : *ptrSpeed;`
`void (Vehicle: : *ptrDisplay) () const;`

b. input:

```
#include <iostream>
```

```
class Vehicle {  
public:
```

```
    int speed;  
    float fuelLevel;
```

```
    Vehicle(int spd, float fuel) : speed(spd), fuelLevel(fuel) {}
```

```
    void displayStatus() const {  
        std::cout << "Speed: " << speed << " km/h, Fuel Level: " << fuelLevel << " liters\n";  
    }  
};
```

```
int main() {  
    Vehicle car(0, 50.0f);
```

```
    int Vehicle::*ptrSpeed = &Vehicle::speed;
```

```
    void (Vehicle::*ptrDisplay)() const = &Vehicle::displayStatus;
```

```
    car.*ptrSpeed = 80;
```

```
    (car.*ptrDisplay)();
```

```
    return 0;  
}
```

output:

Speed: 80 km/h, Fuel Level: 50 liters

c. Uninitialized Member Pointers

If a pointer to a member is used without initialization, it results in undefined behavior.

Mitigation: Always initialize member pointers before use.

Using Pointers from a Null Object

Calling a member function through a pointer on a null object (nullptr) leads to a segmentation fault.

Mitigation: Check if the object pointer is valid before dereferencing.

Complexity and Readability

Pointers to members have a somewhat confusing syntax (*. and -> *).

Mitigation: Use clear naming conventions and comments to improve readability.

Non-Static Members Only

Pointers to members only work with non-static members. Static members require direct pointer access.

Mitigation: Ensure you are using them only for instance members.

2. **Imagine you are designing a Student class. In this class, the student's name should be directly accessible (public), but the id should be private and accessible only through a getter function. (a) How would you declare the Student class with a public name member and a private id member? (b) Provide a code snippet that demonstrates creating a Student object, directly modifying the name, and accessing the id via its getter function. (c) Why might a designer choose to allow direct access to some members while restricting others?**

```
Ans. a. class Student {
public:
    std::string name;

    Student(std::string studentName, int studentId) : name(studentName), id(studentId) {}

    int getId() const {
        return id;
    }

private:
    int id;
};
```

b. input:

```
#include <iostream>
```

```
class Student {
public:
    std::string name;
    Student(std::string studentName, int studentId) : name(studentName), id(studentId) {}

    int getId() const {
        return id;
    }

private:
    int id;
};

int main() {
    Student student("Pushkal", 12345);

    student.name = "Sharma";

    std::cout << "Student Name: " << student.name << std::endl;
    std::cout << "Student ID: " << student.getId() << std::endl;

    return 0;
}
```

output:

```
Student Name: Sharma
Student ID: 12345
```

c. One might keep his data private due to Security, Encapsulation & Data Integrity, Maintenance & Flexibility, and Clarity.

3. Design a C++ class named BankAccount with the following attributes:

accountNumber (integer)

balance (double)

accountHolderName (string)

Methods:

deposit(amount): Adds the specified amount to the balance.

withdraw(amount): Deducts the specified amount from the balance, if sufficient funds are available.

displayBalance(): Displays the current balance.

Write a program that:

Creates a BankAccount object.

Prompts the user to enter account details and initial balance.

Allows the user to perform deposit and withdrawal operations.

Displays the final balance

Ans. input:

```
#include <iostream>
```

```
#include <string>
```

```
class BankAccount {
```

```
private:
```

```
    int accountNumber;
```

```
    double balance;
```

```
    std::string accountHolderName;
```

```
public:
```

```
    BankAccount(int accNum, std::string accName, double initialBalance) {
```

```
        accountNumber = accNum;
```

```
        accountHolderName = accName;
```

```
        balance = (initialBalance >= 0) ? initialBalance : 0.0;
```

```
    }
```

```
    void deposit(double amount) {
```

```
        if (amount > 0) {
```

```
            balance += amount;
```

```
            std::cout << "Deposited: $" << amount << "\n";
```

```
        } else {
```

```
            std::cout << "Invalid deposit amount.\n";
```

```
        }
```

```
    }
```

```
    void withdraw(double amount) {
```

```
        if (amount > 0 && amount <= balance) {
```

```
            balance -= amount;
```

```
            std::cout << "Withdrawn: $" << amount << "\n";
```

```
        } else if (amount > balance) {
```

```
            std::cout << "Insufficient funds.\n";
```

```
        } else {
```

```
            std::cout << "Invalid withdrawal amount.\n";
```

```
        }
```

```
    }
```

```

void displayBalance() const {
    std::cout << "Current Balance: $" << balance << "\n";
}
};

int main() {
    int accNumber;
    std::string accHolderName;
    double initialBalance;

    std::cout << "Enter Account Number: ";
    std::cin >> accNumber;
    std::cin.ignore();

    std::cout << "Enter Account Holder Name: ";
    std::getline(std::cin, accHolderName);

    std::cout << "Enter Initial Balance: ";
    std::cin >> initialBalance;

    BankAccount myAccount(accNumber, accHolderName, initialBalance);

    int choice;
    double amount;

    do {

        std::cout << "\nBanking Menu:\n";
        std::cout << "1. Deposit\n";
        std::cout << "2. Withdraw\n";
        std::cout << "3. Display Balance\n";
        std::cout << "4. Exit\n";
        std::cout << "Enter your choice: ";
        std::cin >> choice;

        switch (choice) {
            case 1:
                std::cout << "Enter amount to deposit: ";
                std::cin >> amount;
                myAccount.deposit(amount);
                break;
            case 2:
                std::cout << "Enter amount to withdraw: ";
                std::cin >> amount;

```

```

        myAccount.withdraw(amount);
        break;
    case 3:
        myAccount.displayBalance();
        break;
    case 4:
        std::cout << "thanks for using this banking atm!\n";
        break;
    default:
        std::cout << "Invalid choice. Please try again.\n";
    }
} while (choice != 4);

return 0;
}

```

output:

Enter Account Number: 12345
Enter Account Holder Name: Pushkal
Enter Initial Balance: 1000

Banking Menu:

1. Deposit
2. Withdraw
3. Display Balance
4. Exit

Enter your choice: 1

Enter amount to deposit: 1000

Deposited: \$1000

Banking Menu:

1. Deposit
2. Withdraw
3. Display Balance
4. Exit

Enter your choice: 2

Enter amount to withdraw: 500

Withdrawn: \$500

Banking Menu:

1. Deposit
2. Withdraw
3. Display Balance

4. Exit

Enter your choice: 3

Current Balance: \$1500

Banking Menu:

1. Deposit

2. Withdraw

3. Display Balance

4. Exit

Enter your choice: 4

thanks for using this banking atm!

4. Design a C++ class named Book with the following attributes:

title (string)

author (string)

isbn (string)

available (boolean)

Design a class named Library with the following attributes:

books (an array or vector of Book objects)

Methods:

addBook(book): Adds a new book to the library.

searchBookByTitle(title): Searches for a book based on its title.

borrowBook(isbn): Marks a book as unavailable if it's available.

returnBook(isbn): Marks a book as available.

Ans. #include <iostream>

#include <vector>

#include <string>

class Book {

public:

std::string title;

std::string author;

std::string isbn;

bool available;

Book(std::string t, std::string a, std::string i) : title(t), author(a), isbn(i), available(true) {}

void display() const {

std::cout << "Title: " << title << "\nAuthor: " << author << "\nISBN: " << isbn
<< "\nStatus: " << (available ? "Available" : "Not Available") << "\n\n";

}

};

```

class Library {
private:
    std::vector<Book> books;

public:
    void addBook(const Book& book) {
        books.push_back(book);
        std::cout << "Book \"" << book.title << "\" added to the library.\n";
    }

    void searchBookByTitle(const std::string& title) const {
        for (const auto& book : books) {
            if (book.title == title) {
                book.display();
                return;
            }
        }
        std::cout << "Book titled \"" << title << "\" not found.\n";
    }

    void borrowBook(const std::string& isbn) {
        for (auto& book : books) {
            if (book.isbn == isbn) {
                if (book.available) {
                    book.available = false;
                    std::cout << "Book \"" << book.title << "\" has been borrowed.\n";
                } else {
                    std::cout << "Book \"" << book.title << "\" is already borrowed.\n";
                }
            }
        }
        return;
    }

    std::cout << "Book with ISBN " << isbn << " not found.\n";
}

    void returnBook(const std::string& isbn) {
        for (auto& book : books) {
            if (book.isbn == isbn) {
                if (!book.available) {
                    book.available = true;
                    std::cout << "Book \"" << book.title << "\" has been returned.\n";
                } else {
                    std::cout << "Book \"" << book.title << "\" was not borrowed.\n";
                }
            }
        }
    }
}

```



```

        return;
    }
}
std::cout << "Book with ISBN " << isbn << " not found.\n";
}

void displayAllBooks() const {
    if (books.empty()) {
        std::cout << "No books in the library.\n";
        return;
    }
    std::cout << "Library Collection:\n";
    for (const auto& book : books) {
        book.display();
    }
}
};

int main() {
    Library library;

    library.addBook(Book("Adventures of Tom Sawyer", "Mark Twain", "54321"));
    library.addBook(Book("Time Machine", "H.G. Wells", "09876"));
    library.addBook(Book("Utopia", "Sir Thomas Moor", "13579"));

    int choice;
    std::string title, isbn;

    do {
        std::cout << "\nLibrary Menu:\n";
        std::cout << "1. Search Book by Title\n";
        std::cout << "2. Borrow Book\n";
        std::cout << "3. Return Book\n";
        std::cout << "4. Display All Books\n";
        std::cout << "5. Exit\n";
        std::cout << "Enter your choice: ";
        std::cin >> choice;
        std::cin.ignore();

        switch (choice) {
            case 1:
                std::cout << "Enter book title: ";
                std::getline(std::cin, title);
                library.searchBookByTitle(title);

```

```

        break;
    case 2:
        std::cout << "Enter book ISBN to borrow: ";
        std::cin >> isbn;
        library.borrowBook(isbn);
        break;
    case 3:
        std::cout << "Enter book ISBN to return: ";
        std::cin >> isbn;
        library.returnBook(isbn);
        break;
    case 4:
        library.displayAllBooks();
        break;
    case 5:
        std::cout << "Exiting the library system. Goodbye!\n";
        break;
    default:
        std::cout << "Invalid choice, please try again.\n";
    }
} while (choice != 5);

return 0;
}

```

output:

Book "Adventures of Tom Sawyer" added to the library.
 Book "Time Machine" added to the library.
 Book "Utopia" added to the library.

Library Menu:

1. Search Book by Title
2. Borrow Book
3. Return Book
4. Display All Books
5. Exit

Enter your choice: 1

Enter book title: Adventures of Tom Sawyer

Title: Adventures of Tom Sawyer

Author: Mark Twain

ISBN: 54321

Status: Available

Library Menu:

1. Search Book by Title
2. Borrow Book
3. Return Book
4. Display All Books
5. Exit

Enter your choice: 2

Enter book ISBN to borrow: 09876

Book "Time Machine" has been borrowed.

Library Menu:

1. Search Book by Title
2. Borrow Book
3. Return Book
4. Display All Books
5. Exit

Enter your choice: 3

Enter book ISBN to return: 13579

Book "Utopia" was not borrowed.

Library Menu:

1. Search Book by Title
2. Borrow Book
3. Return Book
4. Display All Books
5. Exit

Enter your choice: 4

Library Collection:

Title: Adventures of Tom Sawyer

Author: Mark Twain

ISBN: 54321

Status: Available

Title: Time Machine

Author: H.G. Wells

ISBN: 09876

Status: Not Available

Title: Utopia

Author: Sir Thomas Moor

ISBN: 13579

Status: Available

Library Menu:

1. Search Book by Title
2. Borrow Book
3. Return Book
4. Display All Books
5. Exit

Enter your choice: 5

Exiting the library system. Goodbye!